

Laboratório de Concorrência · Aula 10

Vinícius Jientara Cabral · RA 24191501-2 · Turma C

Gerado em 09/10/2026, 20:48:20 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de cooperação é quando uma tarefa precisa esperar a outra terminar algo para poder continuar: a ordem entre elas importa. Exemplo do dia a dia: o garçom só pode servir o prato depois que o cozinheiro terminar de prepará-lo.

Sincronização de competição é quando várias tarefas querem usar o mesmo recurso, que não pode ser usado por duas ao mesmo tempo, e o importante é que só uma use por vez (a ordem não importa). Exemplo: duas pessoas querendo usar o mesmo caixa eletrônico ou o mesmo banheiro: uma espera a outra sair.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Esperava 2.000.000, porque cada thread soma 1 um milhão de vezes (1.000.000 + 1.000.000).

1ª execução (n = 1.000.000)

1042425

2ª execução (n = 1.000.000)

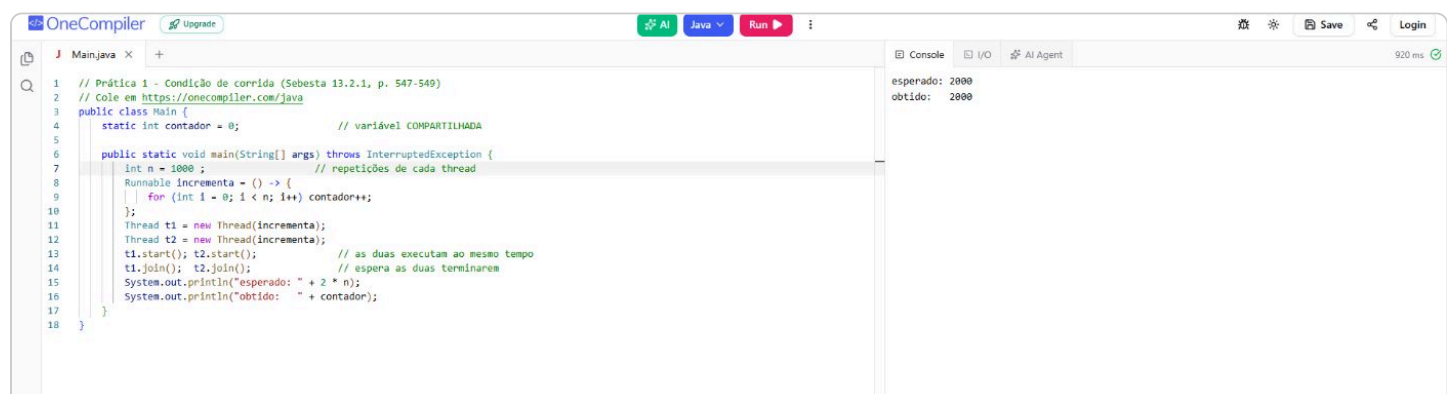
1034854

3ª execução (n = 1.000.000)

1068510

Saída do programa

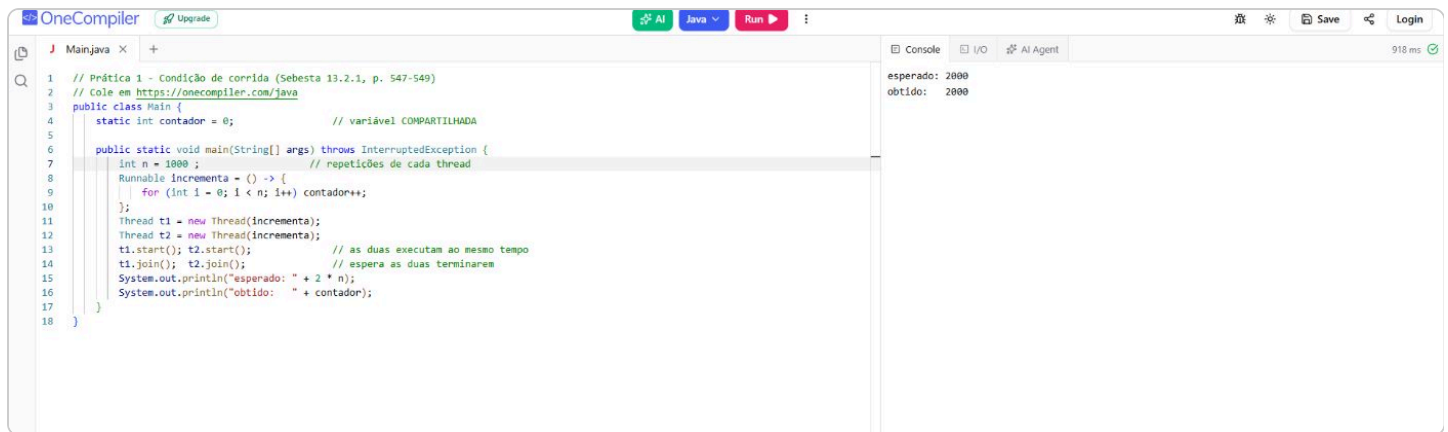
não respondido



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

esperado: 2000
obtido: 2000



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable Incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(Incrementa);
12        Thread t2 = new Thread(Incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

```
esperado: 2000
obtido: 2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram: o esperado era 2.000.000 e todas as execuções com $n = 1.000.000$ deram menos que isso. Os valores também foram diferentes entre si (1042425, 1034854 e 1068510). Mudam porque o escalonador decide a cada execução quando cada thread roda e quando é interrompida, então a quantidade de atualizações que se perdem muda a cada vez.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

`contador++` são três passos: ler, somar e gravar. Exemplo com `contador = 5`:

- 1) A lê 5.
- 2) B lê 5 (A ainda não gravou).
- 3) A soma e grava 6.
- 4) B soma 1 ao 5 que tinha lido e grava 6.

O contador deveria estar em 7, mas ficou em 6: a atualização de A foi sobrescrita e se perdeu. Isso acontece milhares de vezes ao longo de um milhão de repetições

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$ deu 2000 nas duas vezes, ou seja, o erro não apareceu. Cada thread termina tão rápido que uma acaba antes de a outra começar a atrapalhar. Isso é perigoso porque o programa passa nos testes pequenos e parece correto, e o erro só aparece em produção, com carga alta, de forma intermitente e difícil de reproduzir. Um teste que passa não prova que não há condição de corrida.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
ABABAB → 6
AABBAB → 6
BBBAAA → 7
BABAAB → 6
BBAABA → 4

Saída do programa

não respondido

```

1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19            passo[t] += 1
20    return total
21
22
23 print("AABBBA ->", executa("AABBBA"))
24 print("BBBAAA ->", executa("BBBAAA"))
25 print("ABABAB ->", executa("ABABAB"))
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")

```

Console output:

```

AABBBA -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBA
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AABBBA

```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

AAABBB -> 8

BBBAAA -> 7

ABABAB -> 6

Ordens possíveis: 20

TOTAL = 4: 9 ordens, por exemplo AABBBA

TOTAL = 6: 9 ordens, por exemplo AABABB

TOTAL = 7: 1 ordens, por exemplo BBBAAA

TOTAL = 8: 1 ordens, por exemplo AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem AABBBA:

1) A lê TOTAL = 3.

2) A calcula $3 + 1 = 4$ (só no registrador).

3) B lê TOTAL = 3 (A ainda não gravou).

4) B calcula $3 \times 2 = 6$.

5) B grava TOTAL = 6.

6) A grava TOTAL = 4, por cima do 6.

Resultado final: 4. A operação de B foi perdida, porque A gravou um valor calculado a partir do TOTAL antigo (3) e sobrescreveu o resultado de B.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Achei que a versão sem sincronização seria a mais rápida, mas que poderia dar um valor errado. Achei que a `synchronized` seria a mais lenta e a `AtomicInteger` ficaria no meio, ambas com o valor correto.

Saída do programa

não respondido

The screenshot shows the OneCompiler IDE with a Java file named Main.java. The code implements a test for synchronization using a semaphore (semCuidado), a synchronized block (incSync), and AtomicInteger. The main method runs three parallel tasks (ms1, ms2, ms3) and prints the final state of the semaphore, the synchronized block, and the AtomicInteger. The console output shows the results of the execution.

```

1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0; comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

Console Output:

```

esperado: 2000000
sem sincronizacao: 1818957 em 2 ms
synchronized: 2000000 em 122 ms
AtomicInteger: 2000000 em 71 ms

```

The screenshot shows the OneCompiler IDE with the same Java code as the previous screenshot. The console output shows the results of the execution.

```

esperado: 2000000
sem sincronizacao: 1631728 em 4 ms
synchronized: 2000000 em 81 ms
AtomicInteger: 2000000 em 25 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto nas duas execuções. A versão sem sincronização errou na execução 1 (1018957) e na 2 (1631728). Quanto ao tempo, a sem sincronização foi a mais rápida (2 e 4 ms), a AtomicInteger ficou no meio (71 a 25 ms) e a synchronized foi a mais lenta (122 a 81 ms, com grande variação). A versão correta é mais lenta porque há custo para garantir a exclusão mútua: pegar e soltar o lock, threads que ficam esperando a outra e perda de paralelismo. A atômica é mais barata que o lock porque usa uma instrução do processador, sem bloquear a thread.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. Cada AtomicInteger garante que uma operação sozinha (tirar de uma conta ou colocar na outra) é atômica, mas a transferência são duas operações. Entre o débito e o crédito, outra thread pode ver o dinheiro "sumido" (o total do banco fica inconsistente), e a verificação "tem saldo?" seguida do débito também pode sofrer corrida. É preciso tornar a transferência inteira uma seção crítica única, por exemplo com synchronized, um lock ou uma transação de banco de dados, cobrindo as duas contas e sempre travando na mesma ordem para não causar impasse.

Prática 4: Impasse (deadlock)

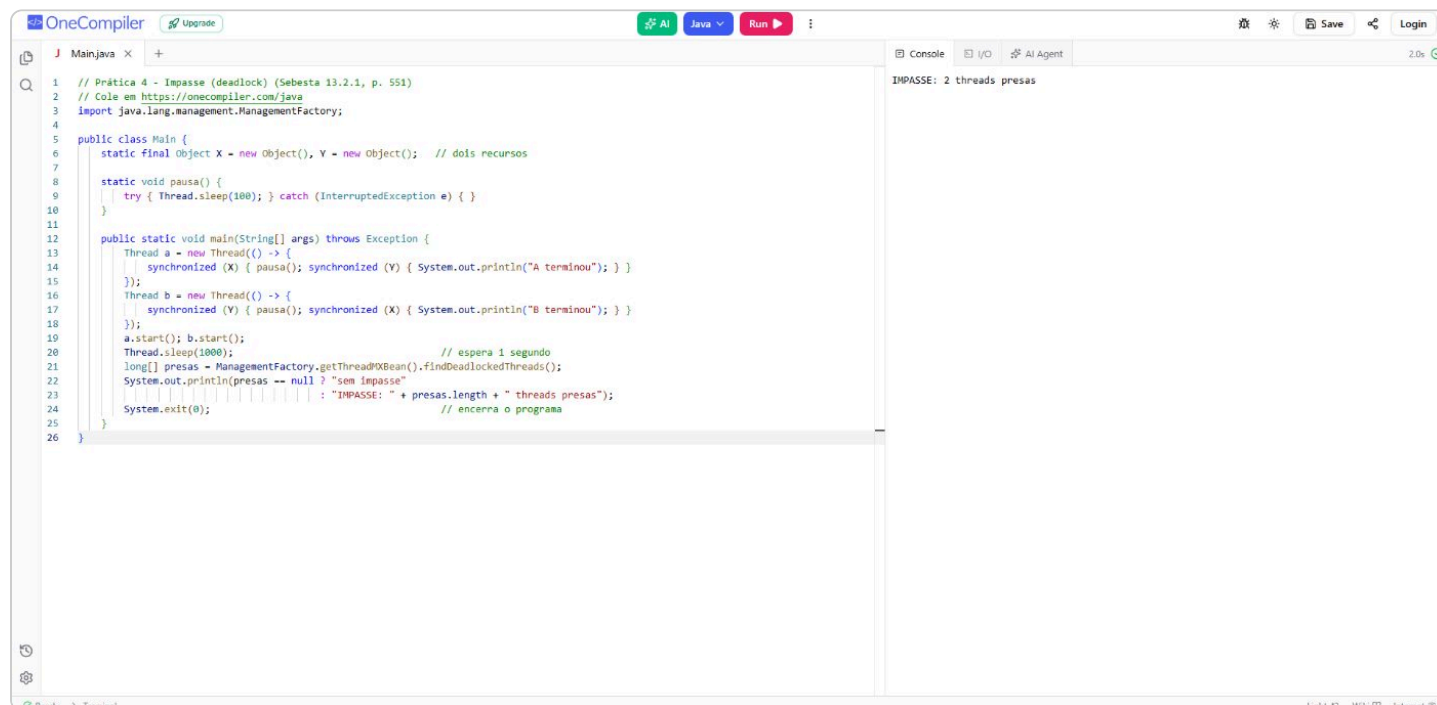
Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que não vão aparecer: A pega X e espera Y, B pega Y e espera X, então as duas ficam presas. Deve imprimir que há impasse.

Saídas (antes e depois da correção)

Antes da correção:
IMPASSE: 2 threads presas

Depois da correção (B pega X e depois Y):
A terminou
B terminou
sem impasse



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse" : "IMPASSE: " + presas.length + " threads presas");
23        System.exit(0); // encerra o programa
24    }
25 }
26 }
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pegou o recurso X e ficou esperando o Y. Ao mesmo tempo, a thread B pegou o Y e ficou esperando o X. Cada uma segura o que a outra precisa e só soltaria depois de conseguir o segundo recurso, então nenhuma avança nunca. O programa detectou 2 threads presas e nenhuma das mensagens "A terminou" e "B terminou" apareceu.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todas as threads pegam os recursos na mesma ordem (primeiro X, depois Y), quem pegou X primeiro é a única que consegue chegar ao Y; a outra espera no X sem segurar nada, então não existe espera circular e a primeira termina e libera os dois. Exemplo real: uma transferência bancária em que a thread 1 transfere da conta A para a B (trava A e depois B) enquanto a thread 2 transfere de B para A (trava B e depois A). Travar sempre a conta de menor número primeiro evita o impasse.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, 200 ms (duas rodadas de 3 tarefas de 100 ms). Com 1 trabalhador, 600 ms (6 tarefas em sequência). Com 6 trabalhadores, cerca de 100 ms (todas ao mesmo tempo).

Tempo com 1 trabalhador

600 ms (6 tarefas em sequência)

Tempo com 3 trabalhadores

200 ms (duas rodadas de 3 tarefas de 100 ms).

Tempo com 6 trabalhadores

100 ms (todas ao mesmo tempo).

Saída do programa

não respondido

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

trabalhador 1 fez a tarefa 1
trabalhador 2 fez a tarefa 2
trabalhador 3 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 2 fez a tarefa 5
trabalhador 3 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador as 6 tarefas de 100 ms rodam uma depois da outra: 600 ms. Com 3, são duas rodadas de 3 tarefas em paralelo: 200 ms. Com 6, todas rodam ao mesmo tempo: 100 ms. O tempo cai na proporção do número de trabalhadores, até o limite de 6 tarefas. O programa não precisa de synchronized nem de trava porque nenhuma variável é compartilhada: cada tarefa é entregue a um único trabalhador por mensagem pelo canal, e o próprio canal cuida da sincronização.

Saídas (antes e depois da troca)

não respondido

The screenshot shows the OneCompiler interface with a Go program. The code defines a channel 'ch' with no buffer and attempts to send a value to it. The console output shows a fatal error: 'fatal error: all goroutines are asleep - deadlock!'. The error details indicate that goroutine 1 is stuck in a send operation on channel 'ch'.

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
/box/main.go:9 +0x38
exit status 2

Debug this error

The screenshot shows the same Go program as above, but with a modification: the channel 'ch' is now created with a buffer of size 1. The console output shows the program executed successfully, printing the value 1.

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1)       // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

1

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem buffer, o envio (`ch <- 1`) só termina quando outra goroutine estiver recebendo. Como só existe a goroutine `main` e ela está parada no envio, nunca chega ao recebimento na linha seguinte, e o Go detecta o impasse ("all goroutines are asleep"). Com espaço para 1 mensagem, o envio deixa o valor guardado no buffer e continua sem esperar ninguém, então o recebimento da linha seguinte lê o 1 e o programa termina.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de `main`. O que você acha que acontece?

Achei que talvez desse erro de compilação, porque a thread tenta alterar uma variável que pertence a `main` e o Rust é rígido com isso. Se compilasse, esperava que imprimisse 1.

Saídas (o erro e o resultado com `move`)

não respondido

The screenshot shows the OneCompiler interface with a Rust program that attempts to share a mutable variable between threads. The code is as follows:

```

1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }

```

The console displays several error messages related to borrowing rules:

- `note: function requires argument type to outlive 'static'`
- `error[E0502]: cannot borrow 'contador' as immutable because it is also borrowed as mutable`
- `error: aborting due to 2 previous errors`

Some errors have detailed explanations: E0373, E0502. For more information about an error, try `rustc --explain E0373`.

The screenshot shows the same Rust code as before, but with the `move` keyword added to the thread spawning function:

```

1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }

```

The console now shows warnings instead of errors:

- `warning: value captured by 'contador' is never read`
- `warning: value assigned to 'contador' is never read`
- `warning: 2 warnings emitted`

Saída do programa

não respondido

The screenshot shows a more complex Rust program using `Arc` and `Mutex` to share a counter between threads:

```

1 // Prática 6b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10        let c = Arc::clone(&contador);
11        threads.push(thread::spawn(move || {
12            for _ in 0..100_000 {
13                *c.lock().unwrap() += 1; // trava, soma e destrava
14            }
15        }));
16    }
17    for t in threads { t.join().unwrap(); }
18    println!("contador = {}", *contador.lock().unwrap());
19 }

```

The console output shows the final result of the program:

```

contador = 400000

```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O tipo i32 é copiado, não movido. Com move, a thread leva uma cópia do número e soma 1 na cópia dela. O contador original, que é o de main, nunca é alterado, então imprime 0. O compilador até avisa ("value assigned to contador is never read"), mas o programa compila porque agora não há compartilhamento: ele só deixa de fazer o que eu queria.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro só foi descoberto na execução (e de forma intermitente: com $n = 1000$ nem apareceu). Em Rust foi descoberto na compilação: o compilador recusou o código que compartilhava a variável sem proteção. A vantagem é que a condição de corrida não chega a ir para produção, porque o programa nem compila. O custo para quem programa é uma curva de aprendizado maior e mais cerimônia no código (Arc, Mutex, move, lock().unwrap()), além de o compilador rejeitar programas até se entender o que ele pede.

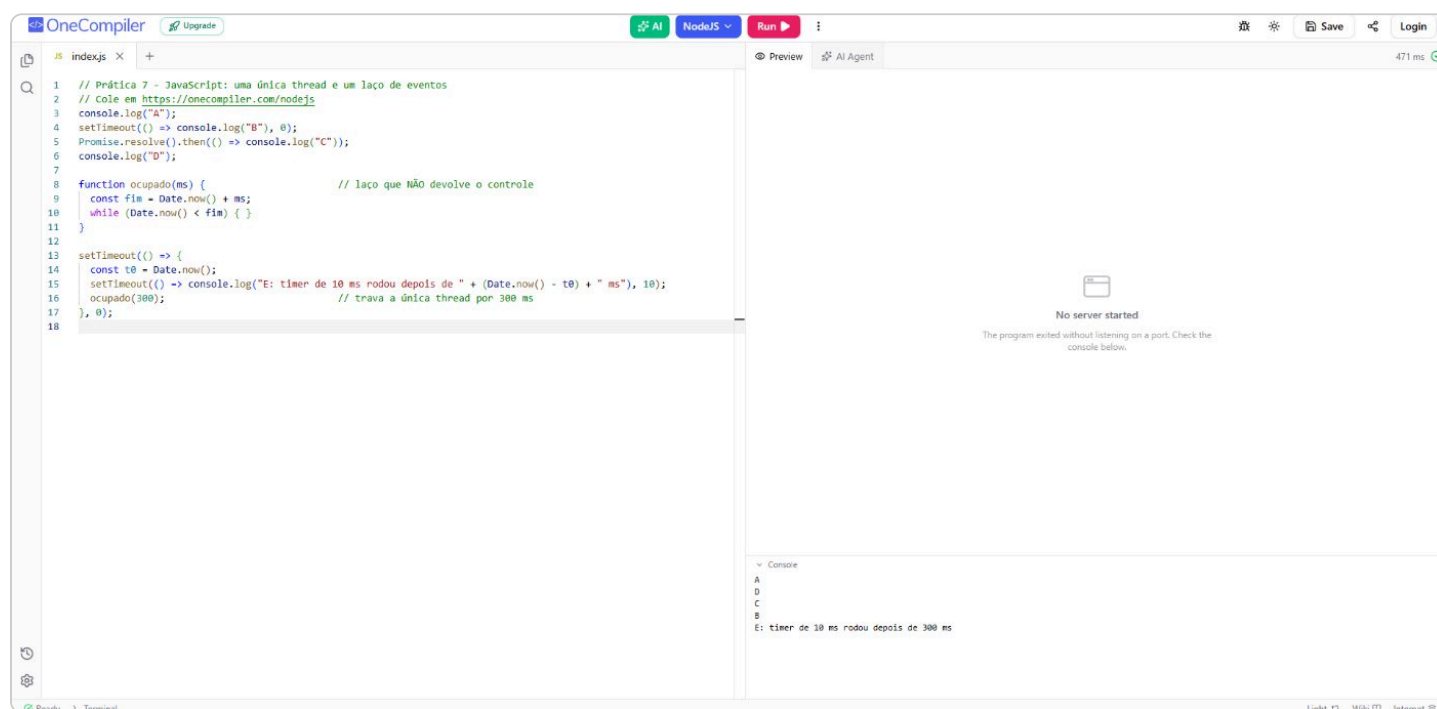
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B: A e D são síncronos e saem primeiro; depois a promessa (C); por último o setTimeout (B).

Saída do programa

não respondido



The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code implements a single-threaded event loop simulation. It logs 'A' and 'D' synchronously, then schedules 'C' as a Promise and 'B' as a setTimeout(0). A function 'ocupado(ms)' is used to simulate a busy thread for 300ms. The output in the console shows the execution order: A, D, C, B, and then a log from the 10ms timer (E) after 300ms.

```
1 // Prática 7 - JavaScript: uma única thread e um laço de eventos
2 // Cole em https://onecompiler.com/nodejs
3 console.log("A");
4 setTimeout(() => console.log("B"), 0);
5 Promise.resolve().then(() => console.log("C"));
6 console.log("D");
7
8 function ocupado(ms) { // laço que NÃO devolve o controle
9   const fim = Date.now() + ms;
10  while (Date.now() < fim) { }
11 }
12
13 setTimeout(() => {
14   const t0 = Date.now();
15   setTimeout(() => console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0) + " ms"), 10);
16   ocupado(300); // trava a única thread por 300 ms
17 }, 0);
```

Console output:

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B (e depois E). A e D são console.log comuns e saem primeiro, na execução síncrona. Depois que o código atual termina, o laço de eventos esvazia a fila de microtarefas, onde está a promessa (C), e só depois pega a próxima tarefa da fila de timers, onde está o B. O setTimeout com tempo 0 não executa na hora: ele só agenda o callback para depois do código atual e das microtarefas.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms só rodou depois de 300 ms porque a única thread estava ocupada no laço ocupado(300), que não devolve o controle ao laço de eventos; o callback do timer só pode ser executado quando o código atual terminar.

Em um servidor Node.js, um cálculo pesado trava a única thread e todos os usuários ficam sem resposta até ele acabar (latência alta para todos). Para evitar isso, o trabalho pesado deve ser quebrado em pedaços, ir para worker threads ou processos separados, ou para outro serviço.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida do tipo verificar e depois agir: as duas requisições leem "1 lugar disponível" antes de qualquer uma gravar e as duas vendem o mesmo ingresso (venda em duplicidade), como as atualizações perdidas da Prática 1. Para evitar, a verificação e a reserva devem ser uma operação atômica: por exemplo, uma transação no banco com lock da linha (SELECT ... FOR UPDATE) ou um UPDATE condicional (UPDATE ... SET vendido = true WHERE id = ? AND vendido = false) conferindo quantas linhas foram alteradas, além de uma restrição de unicidade por assento.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Que o erro some com testes pequenos: com $n = 1000$ o programa Java sempre deu certo, e até a versão sem sincronização acertou em uma das duas execuções da Prática 3, então o código errado pode parecer correto.